
C-EASY-2：指针

Step 1. 指针与结构体

什么是指针？

1.如何在C语言中定义指针变量？指针变量的大小是固定的吗？其大小与什么有关？

通过 类型 * 指针名 的形式定义，其中 * 表示变量是指针。

指针变量的大小是固定的，与类型无关。

指针变量的大小由cpu的位数决定，32位对应4字节，64位对应8字节。

2.写出以下代码的输出结果，并解释原因。

```
int x = 10;
int* p = &x; //x的地址给p
*p = 20; //将p地址上对应的值赋值为20，x因此变成了20

int arr[3] = {3, 6, 9};
int *q = arr; //arr的 首元素地址 给q，此时*q为3
int y = ++*arr + *++q; /* 查询资料可知 *和++优先级相同，遵循从右往左。前者可以理解为对 arr的首元素地址解引用获得第一个元素的值,再通过++赋值最终第一个元素的值变成了4；后者可以理解为对q的地址进行加一操作，而q的地址加1就是arr[1]的地址，对其进行解引用得到了6。y最终为10。 */

printf("%d %d", x, y);
```

根据以上分析最终输出结果为：

20 10

3.什么是野指针？简述其危害。如何避免产生野指针？

野指针就是指向一个非法内存地址的指针，这个存储的地址是无效的。举例：因为指针没有初始化导致存储了随机的地址。

危害：可能造成程序崩溃或者篡改未知的内存，同时因为编译器不会报错也可能导致程序运行正常但是得不到期望结果，增加纠错成本。

避免方法：

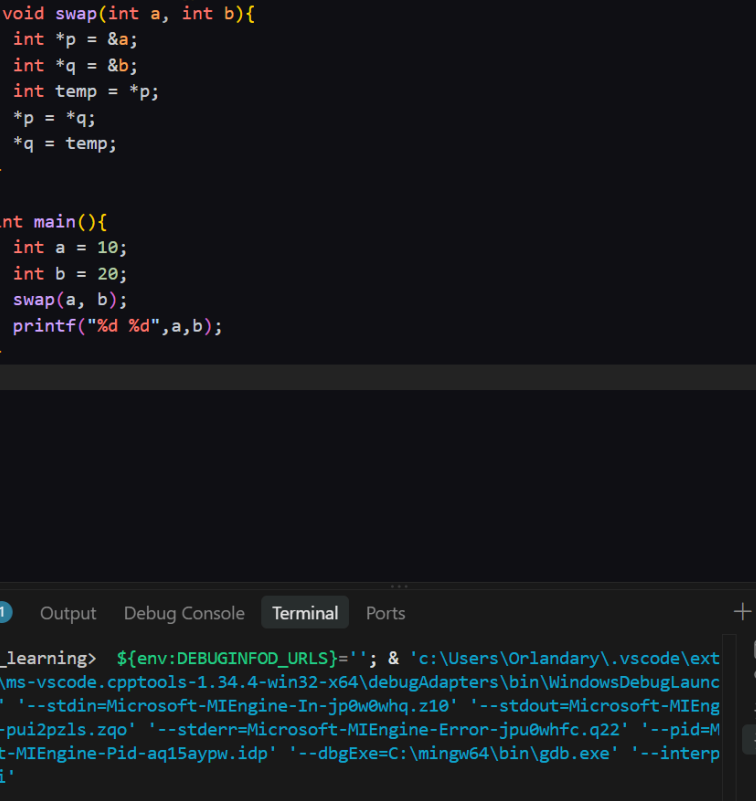
1. 初始化时对指针赋值为NULL；
2. 在free完内存后记得NULL一下指针；
3. 小心指针越界，特别注意使用数组时。

4.尝试设计一个真正有效的swap()函数

错误函数如下，现尝试在此基础上修改调试

```
void swap(int a, int b){  
    int temp = a;  
    a = b;  
    b = temp;  
}  
  
int main(){  
    int a = 10;  
    int b = 20;  
    swap(a, b);  
}
```

第一次尝试如下：



The image shows a VS Code editor window with a C program named `main.c`. The program defines a `swap` function and a `main` function. The `main` function initializes `a = 10` and `b = 20`, calls `swap(a, b)`, and prints the values. The terminal output shows the program running successfully, printing `10 20`.

```
C main.c 1 X C part3.c X c_cpp_properties.json X launch.json X tasks.json X D v D C/C++: ...  
C main.c > ...  
1  #include <stdio.h>  
2  void swap(int a, int b){  
3      int *p = &a;  
4      int *q = &b;  
5      int temp = *p;  
6      *p = *q;  
7      *q = temp;  
8  }  
9  
10 int main(){  
11     int a = 10;  
12     int b = 20;  
13     swap(a, b);  
14     printf("%d %d",a,b);  
15 }  
16  
17  
18  
  
Problems 1 Output Debug Console Terminal Ports  
PS D:\c_learning> $(env:DEBUGINFOD_URLS=''; & 'C:\Users\Orlandary\.vscode\extensions\ms-vscode.cpptools-1.34.4-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-jp0w0whq.z10' '--stdout=Microsoft-MIEngine-Out-pui2pzls.zqo' '--stderr=Microsoft-MIEngine-Error-jpu0whfc.q22' '--pid=Microsoft-MIEngine-Pid-aq15aypw.idp' '--dbgExe=C:\mingw64\bin\gdb.exe' '--interpreter=mi')  
10 20  
PS D:\c_learning>
```

思考：没有抓住本质，实参的地址才是关键，对函数输入做些改变
更改后如下

```
C main.c X C part3.c X {} c_cpp_properties.json X {} launch.json X {} tasks.json X
C main.c > swap(int *, int *)
1  #include <stdio.h>
2  void swap(int *p, int *q){
3      int temp = *p;
4      *p = *q;
5      *q = temp;
6  }
7
8  int main(){
9      int a = 10;
10     int b = 20;
11     swap(&a, &b);
12     printf("%d %d", a, b);
13 }
14
15
16

Problems Output Debug Console Terminal Ports
PS D:\c_learning> ${env:DEBUGINFOD_URLS}=''; & 'c:\Users\Orlandary\.vscode\extensions\ms-vscode.cpptools-1.34.4-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-jp0w0whq.z10' '--stdout=Microsoft-MIEngine-Out-pui2pzls.zqo' '--stderr=Microsoft-MIEngine-Error-jpu0whfc.q22' '--pid=Microsoft-MIEngine-Pid-aq15aypw.idp' '--dbgExe=C:\mingw64\bin\gdb.exe' '--interpreter=mi'
10 20
PS D:\c_learning> ^C
PS D:\c_learning>
PS D:\c_learning> ${env:DEBUGINFOD_URLS}=''; & 'c:\Users\Orlandary\.vscode\extensions\ms-vscode.cpptools-1.34.4-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-u2gxrcsj.htx' '--stdout=Microsoft-MIEngine-Out-sewmdcb1.b3f' '--stderr=Microsoft-MIEngine-Error-faxvflrm.ohd' '--pid=Microsoft-MIEngine-Pid-3xqklbt5.2ev' '--dbgExe=C:\mingw64\bin\gdb.exe' '--interpreter=mi'
20 10
PS D:\c_learning>
```

什么是结构体？

1.请你完成一个PerInfo结构体的定义，成员组成如下。了解一下typedef关键字与结构体的一般用法，利用typedef为你刚刚定义的结构体取一个别名。提交最终的结构体定义。

1. 个人姓名（字符型数组，长度为10个字节）
2. 性别（字符型）
3. 年龄（整型）
4. 身高（双精度浮点型）

一个完整的结构体应该遵循以下模板：

```
struct 结构体名 {  
    成员类型1 成员名1;  
    成员类型2 成员名2;  
    // ...更多成员  
}; // 注意：分号不能丢！
```

完成PerInfo定义

```
struct PerInfo{  
    char name[10];  
    char gender;  
    int age;  
    double height;  
  
} ;
```

typedef可以给已存在的数据类型起别名。

在这里的作用是可以给已存在的结构体取别名。

```
struct PerInfo{  
    char name[10];  
    char gender;  
    int age;  
    double height;  
}  
typedef struct PerInfo personalinformation; //此时的别名可以单独使用
```

2.了解并说明结构体指针的含义。

通过下面这个列子理解

其中的s是结构体的变量，类似于int 后面定义的东西。

s的大小是sizeof这个结构体。

```
struct PerInfo s;  
struct PerInfo *p = &s; // p是指针，存放s的首地址
```

指针p的类型是 struct PerInfo *,p中存放的是s的起始地址。

与普通指针类似的，该结构体指针本身大小只与系统位数有关（32 位系统 4 字节，64 位系统 8 字节）。

对于访问成员：

- s用s.age= 的形式
- 指针p用p->age 的形式

3.了解一下结构体的内存对齐规则，据此计算一下你刚刚定义的结构体占用字节的大小（可用sizeof运算符验证计算结果）。提交计算过程。（此外，你还可以尝试更改一下结构体中成员的定义顺序，看看对结构体占用字节数的影响）

对齐规则：

- 对于每一个成员的起始地址，必须是成员自身对齐值的整数倍。
- 结构体的整体大小一定是最大成员大小的整数倍。
- 顺序会影响占用字节，必须严格按照代码书写顺序。

```
struct PerInfo{  
    char name[10];  
    char gender;  
    int age;  
    double height;  
  
};
```

计算过程如下：

char 1字节，int 4字节，double 8字节。

char name[10] 占了0到9的地址，char gender占了10的位置；

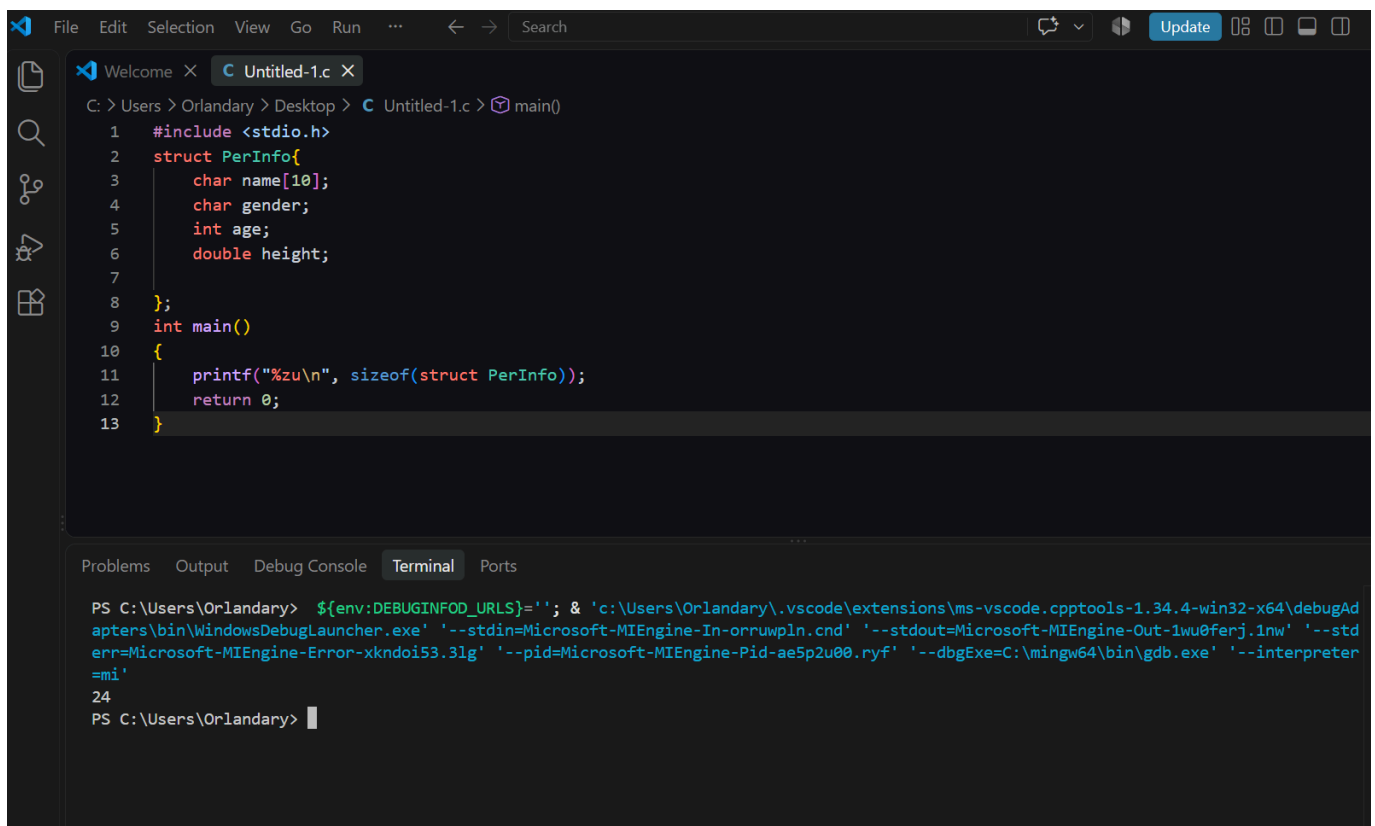
根据规则第一条，int要在4的倍数12，因此padding在11，int age占12，13，14，15；

此时16恰好为height的两倍，height占到15+8=23；

用第二规则检验一下，一共24，是8的倍数，无需再填充padding。

验证：

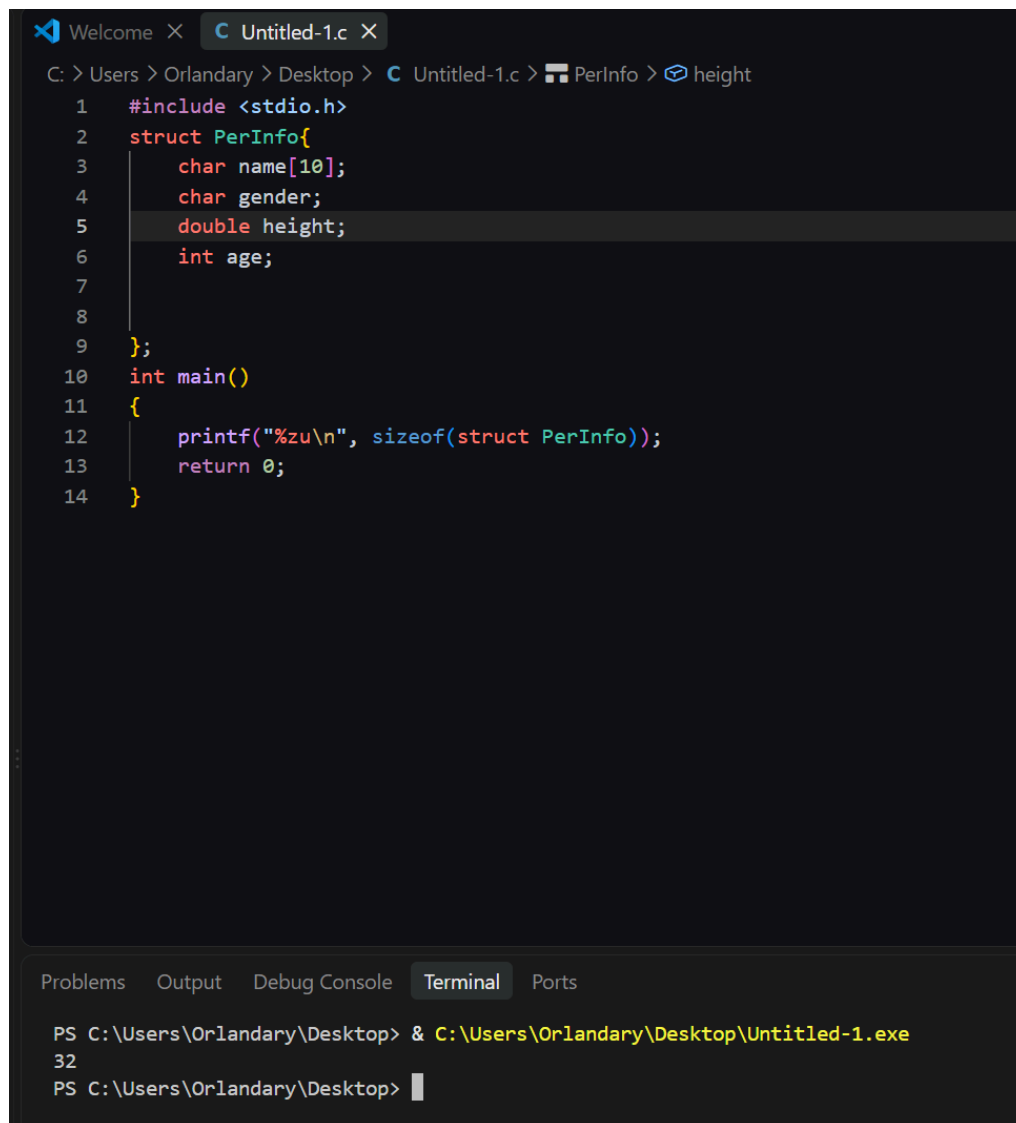
```
#include <stdio.h>
struct PerInfo{
    char name[10];
    char gender;
    int age;
    double height;
};
int main()
{
    printf("%zu\n", sizeof(struct PerInfo));
    return 0;
}
```



The screenshot shows the Visual Studio Code (VS Code) interface. The editor window displays the C code from the previous block. The file explorer on the left shows the file 'Untitled-1.c'. The terminal window at the bottom shows the command prompt output. The command executed is a complex command to run the program with debug options. The output shows the size of the struct PerInfo as 24 bytes.

```
PS C:\Users\Orlandary> ${env:DEBUGINFOD_URLS}=''; & 'c:\Users\Orlandary\.vscode\extensions\ms-vscode.cpptools-1.34.4-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-orrwupln.cnd' '--stdout=Microsoft-MIEngine-Out-1wu0ferj.1nw' '--stderr=Microsoft-MIEngine-Error-xkndoi53.3lg' '--pid=Microsoft-MIEngine-Pid-ae5p2u00.ryf' '--dbgExe=C:\mingw64\bin\gdb.exe' '--interpreter=mi'
24
PS C:\Users\Orlandary>
```

尝试交换顺序结果如下：



```
1  #include <stdio.h>
2  struct PerInfo{
3      char name[10];
4      char gender;
5      double height;
6      int age;
7
8
9  };
10 int main()
11 {
12     printf("%zu\n", sizeof(struct PerInfo));
13     return 0;
14 }
```

Problems Output Debug Console Terminal Ports

```
PS C:\Users\Orlandary\Desktop> & C:\Users\Orlandary\Desktop\Untitled-1.exe
32
PS C:\Users\Orlandary\Desktop>
```

Step2. 链操作

什么是链表？

1.请对比链表和数组的存储，讲讲链表和数组的区别

数组的存储是地址连续的数据，元素类型统一。

链表的存储各个节点的地址并不是一定相邻的，是取决于指针域，该节点的指针域存的是下一个节点的地址，我理解为一个房间里存在的物品和下一个房间的钥匙。

2. 简述单向链表节点的结构特点。定义一个只存储一个整数的单向链表节点。

每个节点由数据域和指针域组成，指针域存放的是下一个节点的地址，用指针next表示。需要注意的是访问只能通过当前节点的指针向后访问，最后一个节点的指针域置为NULL,表示结束。

定义如下

```
struct Glimmer {
    int cs;
    struct Glimmer *next;
};
```

实现基本的链操作

基础必备步骤(先通过typedef别名成g)

```
g *head;    // 定义头指针

head = (g*)malloc(sizeof(g));
// 1. 申请节点空间 其中(g*)作用是强制转换成节点指针类型, 查阅资料知这里可以省略

head->cs = 0; // 2. 设置数据域 细节指针使用->的符号

head->next = NULL; // 3. 只有这一个节点, 没有下一个节点, next为null
```

典型错误

```
typedef struct Glimmer {
    int cs;
    struct Glimmer *next;
} g;

int main(){
    g *head;
    head= (g*) malloc(sizeof(g));
    head->cs=0;
    head->next=NULL;

}
```

1. 缺少头文件 `#include <stdlib.h>;`
2. 未检查 `malloc` 返回值, 内存分配失败时访问 `head->cs` 会崩溃;
3. 没有调用 `free` 释放堆内存, 造成内存泄漏。

修改后完成基础操作

```

#include <stdlib.h> //头文件
typedef struct Glimmer {
    int cs;
    struct Glimmer *next;
} g;

int main(){
    g *head;
    head= (g*) malloc(sizeof(g));
    if(head == NULL) //判断
    {
        return -1;
    }
    head->cs=0;
    head->next=NULL;

    free(head); // 释放内存
    head=NULL;
    return 0;

}

```

添加元素:

请你设计一个函数，传入一个数据（int即可），返回一个存储这个数据的节点。

模仿头指针的创建过程，另外注意加上数据域

尝试结果如下：

```

g* newnode(int num){
    g* s;
    s=(g*)malloc(sizeof(g));
    if(s == NULL)
    {
        return -1;
    }
    s->cs=num;

    return s;

}

```

检查错误修改

1.返回值不能是-1. 2.加一条next为NULL, 防止垃圾值

3. 对比区分 (选择题常考)

- 如果函数返回类型是 `int` (比如 `main` 函数) : `return -1;` 没问题, 代表程序异常退出。

```
c ^ 运行 复制 分享  
  
int main()  
{  
    if(head == NULL)  
        return -1; // ✓ main返回int, 所以可以return -1  
}
```

这就是你刚才 `main` 函数里可以写 `return -1` 但 `newnode` 函数不能写的根本区别!

`main` 返回 `int`; `newnode` 返回 `g*` 指针!

```
g* newnode(int num){  
    g* s;  
    s=(g*)malloc(sizeof(g));  
    if(s == NULL)  
    {  
        return NULL;  
    }  
    s->data=num;  
    s->next=NULL;  
    return s;  
}
```

请你实现头插函数。函数传入主链表的头指针`head`和一个数据`n`

按照提示初步尝试:

关键点: 定义新`A`; `A`指向头节点; 头指针指向`A`。

```

g* headplus(g* head,int n){
    g* A;
    A=(g*)malloc(sizeof(g));
    if(A==NULL)
    {
        return NULL;
    }
    A->cs=n;
    A->next=head ;

}

```

经过ai纠错

1. 重点错误，还是没有理解提示和头插的逻辑，没有返回值，应该末尾返回新的头节点，相当于创建一个新的head，把老的head和n结合成新节点
2. 可以提升的点：return NULL改成 return head。这样即使错误也能保全外部的head。

```

g* headplus(g* head,int n){
    g* A;
    A=(g*)malloc(sizeof(g));
    if(A==NULL)
    {
        return head;
    }
    A->cs=n;
    A->next=head ;

    return A;
}

```

调用的格式应该是：

```

head = headplus(head, n);

```

请你设计尾插函数，并用代码将其实现。（提示：不要直接用头指针，定义一个新的指针将其赋值为头指针）

```

g* tailplus(g* head,int n){
    g* A;
    A=(g*)malloc(sizeof(g));
    if(A==NULL)
    {
        return head;
    }
    A->cs=n;
    A->next=NULL ;

    g* find=head;

    return A;
}

```

到这基本思路卡壳，无法理解和使用这个定义的新指针的作用也想不出应该怎么return。

借助ai完善理解：

```

g* tailplus(g* head, int n) {
    g* A = (g*)malloc(sizeof(g));
    if (A == NULL) {
        return head;
    }

    A->cs = n;
    A->next = NULL;

    if (head == NULL) {
        return A;
    } // 若为空，可以直接用头插函数的做法，这是必要的

    g* p = head;
    while (p->next != NULL) {
        p = p->next;
    } // 理解：找到当p->next是NULL的节点
    p->next = A; // 把新节点接到刚刚找到的节点尾部

    return head;
    /*头节点没有变，返回原来的 head，所以这个函数实际上应该不需要返回什么东西*/
}

```

调用的格式

```
head = tailplus(head, n);
```

查找元素：

通过遍历思想，我们可以打印链表每个节点的数据。请你实现这个打印链表函数，传入链表头指针。且打印后不要改变头指针的指向。

初步想法： 1.通过NULL判断结束。2.按照上面的思路，引入一个可以随便乱造的p。3.现学现用上面p->next的写法按顺序遍历。

尝试如下：

```
g* search(g* head){
    g* p=head;
    while(p->next!=NULL){
        printf("%d",p->cs);
        p=p->next;
    }
    printf("%d",p->cs);

    return head;
}
```

该代码的问题：

- 1.空链表的p->next会出错。
- 2.循环结构可以简化。
- 3.小问题 打印格式优化。
- 4.不需要返回值。

```
void search(g* head){
    g* p=head;
    while(p !=NULL){
        printf("%d ",p->cs);//加了空格
        p=p->next;
    }
    printf("\n");
    // return head; 不需要返回值
}
```

直接调用就行

初步整合后验证如下

```
C Untitled-1.c X  Untitled-1 X  Untitled-2 X
C: > Users > Orlandary > Desktop > C Untitled-1.c > search(g *)
46 void search(g* head){
52     printf("\n");
53     // return head; 不需要返回值
54 } //打印所有数据
55
56 int main(){
57     g *head;
58     head=(g*)malloc(sizeof(g));
59     if(head == NULL) //判断
60     {
61         return -1;
62     }
63
64     head->cs=0;
65     head->next=NULL;
66
67     head=headplus(head,1);
68     head=tailplus(head,2);
69     search(head);
70
71
72
73     free(head); // 释放内存
74     return 0;
75
Problems  Output  Debug Console  Terminal  Ports
PS C:\Users\Orlandary> ${env:DEBUGINFOD_URLS}=''; & 'c:\Users\Orlandary\.vscode\extensions\ms-vscode.cpptools-1.34.4-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-s3erkwa4.sp1' '--stdout=Microsoft-MIEngine-Out-xmxms0wi.ssc' '--stderr=Microsoft-MIEngine-Error-ot3nonkd.12y' '--pid=Microsoft-MIEngine-Pid-dmypo11xw.fzk' '--dbgExe=C:\mingw64\bin\gdb.exe' '--interpreter=mi'
PS C:\Users\Orlandary> ^C
PS C:\Users\Orlandary>
PS C:\Users\Orlandary> ${env:DEBUGINFOD_URLS}=''; & 'c:\Users\Orlandary\.vscode\extensions\ms-vscode.cpptools-1.34.4-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-4fnskwxz.5x3' '--stdout=Microsoft-MIEngine-Out-dtg1hzrk.dbu' '--stderr=Microsoft-MIEngine-Error-3fnduigm.10k' '--pid=Microsoft-MIEngine-Pid-immtijjh.3ub' '--dbgExe=C:\mingw64\bin\gdb.exe' '--interpreter=mi'
1 0 2
PS C:\Users\Orlandary> 
```

请你实现这个查找函数。传入头节点head和要查找的数据n，输出这个节点距头节点的距离（统计这是第几个节点）。如果有多个数据为n的节点只返回第一个，如果没有查找到则返回false。

1.大体框架依旧沿用之前的遍历框架；2.加入ok判断false；3.使用break防止找到第二个数据为n的节点。

尝试如下：

```

void search(g* head,int n){
    g* p=head;
    int i=1;
    int ok=0;
    while(p !=NULL){
        if(p->cs==n){
            printf("%d ",i);
            ok=1;
            break;
        }
        i++;
        p=p->next;
    }
    if(ok==0){
        printf("false");
    }
}

```

通过ai纠错

1.函数名忘记改了。2.通过直接return可以不用ok判断，精简代码。

修改后如下

```

void lookfor(g* head,int n){
    g* p=head;
    int i=1;
    int ok=0;
    while(p !=NULL){
        if(p->cs==n){
            printf("%d ",i);
            ok=1;
            break;
        }
        i++;
        p=p->next;
    }
    if(ok==0){
        printf("false");
    }
}

```

验证结果如下

```
Problems  Output  Debug Console  Terminal  Ports

1 0 2
PS C:\Users\Orlandary> ^C
PS C:\Users\Orlandary>
PS C:\Users\Orlandary> ${env:DEBUGINFOD_URLS}=''; & 'c:\Users\Orlandary\.vscode\extensions\ms-vscode.cpptools-1.34.4-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '-stdIn=Microsoft-MIEngine-In-v3cvntmc.kca' '--stdout=Microsoft-MIEngine-Out-hxsxmsc4.0ft' '--stderr=Microsoft-MIEngine-Error-0uwx4kgc.fn3' '--pid=Microsoft-MIEngine-Pid-s0u2paig.yor' '--dbgExe=C:\mingw64\bin\gdb.exe' '--interpreter=mi'
1 0 2
2
PS C:\Users\Orlandary> ^C
PS C:\Users\Orlandary>
PS C:\Users\Orlandary> ${env:DEBUGINFOD_URLS}=''; & 'c:\Users\Orlandary\.vscode\extensions\ms-vscode.cpptools-1.34.4-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '-stdIn=Microsoft-MIEngine-In-yae1xktn.sc1' '--stdout=Microsoft-MIEngine-Out-u4egp5xr.udc' '--stderr=Microsoft-MIEngine-Error-0udj3tmq.lwm' '--pid=Microsoft-MIEngine-Pid-y5jyyfk.5p4' '--dbgExe=C:\mingw64\bin\gdb.exe' '--interpreter=mi'
```

删除和更改：

请你实现删除第n个节点的函数。只传入头指针和n，如果链表节点数少于n返回false，反之返回true。

1.按照这个要求我们无法提前知道链表节点数，但是如果结束没有实现删除功能可以直接false，这次试一下return。2.引入第二个指针q。

```
void delete(g* head,int n){
    g* p=head;
    g* q=head->next;
    int i=1;
    while(p !=NULL){
        if(i==n-1){
            p->next=q->next;
            free(q);
            printf("true");
            return ;
        }
        p=p->next;
        q=q->next;
        i++;
    }
    printf("false");
}

}
```

ai修改再重新理解

2. 无法删除第 1 个节点

你的判断是 `if(i == n-1)`，而 `i` 从 1 开始。

当 `n = 1` 时，需要 `i == 0`，永远不成立，所以头节点永远删不掉。

3. 删除头节点需要修改外部的 `head` 指针

你的函数签名是 `void delete(g* head, int n)`，`head` 是值传递。

即使你在函数内部让 `head = head->next`，外部的 `head` 也不会变。

所以如果要支持删除第 1 个节点，函数必须返回新的头指针，或者使用二级指针。

4. `q` 可能为 `NULL` 时被解引用

当链表长度小于 `n` 时，`q` 可能变成 `NULL`，此时如果进入 `if` 分支执行 `q->next` 会崩溃。

```

g* delete(g* head, int n) {
    // 非法情况
    if (head == NULL || n <= 0) {
        printf("false\n");
        return head;
    } // 此时没有节点或者n无意义

    // 特殊情况：删除第 1 个节点
    if (n == 1) {
        g* temp = head; // temp的作用类似于我写的q的作用
        head = head->next;
        free(temp);
        printf("true\n");
        return head; // 直接跳出去修改head
    }

    g* p = head;          // p 指向第 n-1 个节点
    g* q = head->next;     // q 指向第 n 个节点
    int i = 1;

    // 当 q 不为空时继续查找，这里修改成q不为空，巧妙避免最大i=n-1的问题
    while (q != NULL) {
        if (i == n - 1) {
            p->next = q->next;
            free(q);
            printf("true\n");
            return head;
        }
        p = p->next;
        q = q->next;
        i++;
    }

    // 链表长度小于 n
    printf("false\n");
    return head;
}

```

反转函数：

请根据上面双指针的思想，设计一个函数，传入头指针，然后将链表倒置，头指针指向新链表的第一个节点。

对于指针q和上一个指针p，q->next应该等于p，但是这样无法继续操作因为q->next已经发生变化了。这里需要再引入一个新的指针。运用prev, curr, next三个指针实现这个过程。这道题我的卡住的点是当q->next的存储值改变就无法处理了，引入一个新指针next可以完美解决。

```

g* reverse(g* head){
    g* prev=NULL;
    g* curr=head;
    g* next;

    while(curr!=NULL){
        next=curr->next;
        curr->next=prev;
        prev=curr;
        curr=next;
    }

    return prev;//注意注意，此时的curr是NULL，prev是最后一个节点。
}

```

调用：

```
head = reverse(head);
```

最终整合所有函数并测试：

```

C: Untitled-1.c X  Untitled-1 X  Untitled-2 X
C: > Users > Orlandary > Desktop > C: Untitled-1.c > main()
148
149 int main(){
150     g* head;
151     head=(g*)malloc(sizeof(g));
152     if(head == NULL)//判断
153     {
154         return -1;
155     }
156
157     head->cxse0;
158     head->next=NULL;
159
160     head=headplus(head,1);
161     head=talipius(head,2);
162     head=talipius(head,3);
163     search(head);
164     lookfor(head,0);
165     head=delete(head,2);
166     head=reverse(head);
167     search(head);
168
169
170     free_list(head);
171     // 释放内存
172     return 0;
173
174 }

```

```

Problems  Output  Debug Console  Terminal  Ports
2
PS C:\Users\Orlandary> ^C
PS C:\Users\Orlandary>
PS C:\Users\Orlandary> $(env:DEBUGINFO_URLS=''; & 'c:\Users\Orlandary\.vscode\extensions\ms-vscode.cpptools-1.34.4-win32-x64\debugAdapters\bin\WindowsDebugLauncher-
y8hr10.21j' '--stdout=Microsoft-MIEngine-Out-a3gcdwxb.pke' '--stderr=Microsoft-MIEngine-Error-gzfsyfi.ptx' '--pid=Microsoft-MIEngine-Pid-v12jogch.mud' '--dbgExe=C:\V
1 0 2 3
2 true
PS C:\Users\Orlandary> ^C
PS C:\Users\Orlandary>
PS C:\Users\Orlandary> $(env:DEBUGINFO_URLS=''; & 'c:\Users\Orlandary\.vscode\extensions\ms-vscode.cpptools-1.34.4-win32-x64\debugAdapters\bin\WindowsDebugLauncher-
2myeolv.yae' '--stdout=Microsoft-MIEngine-Out-xheknpx.ien' '--stderr=Microsoft-MIEngine-Error-22py31f.xx4' '--pid=Microsoft-MIEngine-Pid-gak2pizf.n8q' '--dbgExe=C:\V
1 0 2 3
2 true
3 2 1
PS C:\Users\Orlandary>

```

其他学习知识：循环释放节点的函数

只freehead并没有完全free所有内存。

```
void free_list(g* head) {  
    while (head != NULL) {  
        g* temp = head;  
        head = head->next;  
        free(temp);  
    }  
}
```

吐槽一下我现在使用的唐氏豆包

- 12: 是 4 的倍数, 但 $12 < 11$? 不, $12 > 11$, 但是! **11 这个位置还空着, 不能跳过 11 直接从 12 开始放 age。**

内存是连续的, 不能跳过偏移 11 不用, 直接把 age 塞到 12。

偏移 11、12、13 都在 gender 之后, age 之前, 这一段整块都属于空白。

数字:

当前空闲起点 = 11

大于等于 11 的 4 的倍数:

4 → 太小

8 → 太小

12 → ≥ 11 ✓, 哎, 这里发现我刚才写错了!! **之前这里是我的笔误!!**

大于等于 11, 最小 4 倍数是 **12**, 不是 16!! 我之前写 16 是错误, 抱歉!

现在验证一遍：

gender 用完在 10，空闲偏移 = 11。

age 类型 int (4)：

≥ 11 ，最小 4 倍数：12。填充 11 号字节。age 放在 12。

age 结束在 15，空闲偏移 = 16。

height 类型 double (8)： ≥ 16 最小 8 倍数 = 16，无需填充。height 放在 16。

整体大小 24，是最大成员 8 的倍数，结束。sizeof=24。

之前那个 32 字节是我算错了，感谢你发现漏洞！